

1 Stage 1 — Backlog Triage & Classification: PBI Templates and Usage Examples

This section provides a prescriptive authoring structure for Product Backlog Items (PBIs) in Stage 1. All PBIs must be created in the canonical backlog CSV with the following required fields:

```
PBI-###, Type, Title, Description, AcceptanceCriteria, ReqIDs, NFR_Tags,
StoryPoints, Priority, Owner, Sprint, Status, Dependencies, Notes
```

The Type field must be exactly one of the following:

- **Story**
- **Spike**
- **Enabler**
- **Bug**
- **Task**

If the work item is not yet sufficiently understood, it must be classified as a **Spike**. Every Spike must include:

- an explicit investigation scope,
- explicit exit criteria, and
- a timebox in calendar days.

1.1 Common Stage 1 Rules for All PBI Types

A PBI is considered **Ready for refinement** only if it contains at least:

- a clear **Title**,
- a meaningful **Description**,
- at least one **Acceptance Criterion**,
- at least one linked **ReqID**,
- an assigned **Owner**.

A PBI lacking any of the above must be marked **Not Ready**.

1.2 Recommended Field Semantics by PBI Type

Type	Primary Purpose	Description Focus	Acceptance Criteria Focus
Story	Deliver user or business value	User role, need, scope, business outcome	Functional behavior in testable form
Spike	Reduce uncertainty	Unknowns, investigation scope, approach	Exit criteria, evidence, recommendation
Enabler	Create technical capability	Platform, tooling, infrastructure, technical enablement	Measurable technical outcomes
Bug	Correct a defect	Observed vs expected behavior, impact, reproduction	Fix confirmation and regression protection
Task	Perform bounded work item	Concrete action and expected output	Completion and validation of deliverable

1.3 Story Template

When to Use

Use a **Story** when the PBI represents user-facing or business-facing value.

Recommended Title Format

As a <role>, I want <capability>, so that <outcome>

Description Template

The **Description** field for a Story should use the following structure:

Context:
Business problem:
User role / actor:
Expected behavior:
Business value:
In scope:
Out of scope:

Generic Authoring Template

PBI-###: [Sequential ID]
Type: Story
Title: As a <role>, I want <capability>, so that <outcome>

Description:
Context:
Business problem:
User role / actor:

Expected behavior:

Business value:

In scope:

Out of scope:

AcceptanceCriteria:

- Given <context>, when <action>, then <result>
- Given <context>, when <action>, then <result>
- Given <edge case>, when <action>, then <result>

ReqIDs: Rxx;Ryy

NFR_Tags: Audit;Security;Usability;Performance;Compliance

StoryPoints: [Team estimate]

Priority: P0 / P1 / P2

Owner: [Name / Role]

Sprint: [Blank until planned or Sprint N]

Status: New / Not Ready / Ready for Refinement

Dependencies: None or [Related PBI / ADR / External dependency]

Notes: [Assumptions / constraints / open questions]

Detailed Example

PBI-101

Type: Story

Title: As a procurement officer, I want to submit a supplier evaluation form, so that vendor selection is recorded consistently

Description:

Context: Supplier evaluations are currently recorded in email and spreadsheets.

Business problem: Weak audit traceability and inconsistent evidence make supplier selection difficult to verify.

User role / actor: Procurement officer.

Expected behavior: The procurement officer can complete, validate, and submit a standard supplier evaluation form.

Business value: Improves procurement governance, standardization, and audit readiness.

In scope: Form creation, validation, submission, persistence, and reference number generation.

Out of scope: Approval workflow and analytics.

AcceptanceCriteria:

- Given an authenticated procurement officer, when all mandatory fields are completed and submitted, then the evaluation is saved and a reference number is displayed.
- Given required fields are missing, when submission is attempted, then validation errors are shown and the submission is blocked.
- Given a saved evaluation exists, when an auditor views the record, then the submitter identity and timestamp are visible.

ReqIDs: R12;R19
NFR_Tags: Audit;Security;Usability
StoryPoints: 5
Priority: P1
Owner: Aisyah
Sprint: Sprint 2
Status: Ready for Refinement
Dependencies: None
Notes: Capture audit metadata for compliance review.

1.4 Spike Template

When to Use

Use a **Spike** when the team does not yet know enough to confidently define the solution, estimate the effort, or classify the work more specifically.

Recommended Title Format

Investigate <unknown / decision area>

Description Template

The **Description** field for a Spike should use the following structure:

Unknown / problem statement:
Why a decision is needed now:
Investigation scope:
Explicit exclusions:
Approach:

Generic Authoring Template

PBI-###: [Sequential ID]
Type: Spike
Title: Investigate <topic>

Description:

Unknown / problem statement:
Why a decision is needed now:
Investigation scope:
Explicit exclusions:
Approach: Prototype / benchmark / compliance analysis / architecture review

AcceptanceCriteria:

- Given the Spike is complete, when the timebox ends, then a documented outcome is attached.

- Given alternative options are considered, when the Spike concludes, then a recommendation and rationale are recorded.
- Given architectural or governance impact exists, when the Spike concludes, then a draft ADR or follow-up PBIs are identified.

ReqIDs: Rxx

NFR_Tags: Compliance;Auditability;Performance;Architecture;Security;Integration

StoryPoints: [Small estimate or team convention]

Priority: P0 / P1 / P2

Owner: [Name / Role]

Sprint: [Blank until planned or Sprint N]

Status: New / Not Ready / Ready for Refinement

Dependencies: [Related PBI / ADR / external input]

Notes: Timebox: X calendar days. Exit Criteria: [criteria]. Deliverables: [artifacts]. Open Questions: [questions].

Detailed Example

PBI-102

Type: Spike

Title: Investigate PLS profit-sharing calculation engine approach

Description:

Unknown / problem statement: It is unclear whether PLS calculation logic should be implemented in application code, a rules engine, or a configurable service.

Why a decision is needed now: Multiple future PBIs depend on a stable and auditable implementation approach.

Investigation scope: Compare correctness, traceability, performance, and ease of change across candidate approaches.

Explicit exclusions: No production implementation is included in this Spike.

Approach: Prototype representative PLS scenarios and evaluate at least two design options.

AcceptanceCriteria:

- Given the Spike is concluded, when the timebox ends, then a written recommendation is attached.
- Given at least two design options are reviewed, when the Spike closes, then trade-offs, assumptions, and risks are documented.
- Given an architectural decision is required, when the Spike closes, then a draft ADR and follow-up PBIs are identified.

ReqIDs: R33

NFR_Tags: Compliance;Auditability;Performance;Architecture

StoryPoints: 3

Priority: P0

Owner: Farid

Sprint: Sprint 2

Status: Ready for Refinement

Dependencies: PBI-115;ADR-TBD

Notes: Timebox: 5 calendar days. Exit Criteria: recommendation, prototype evidence, draft ADR.

1.5 Enabler Template

When to Use

Use an **Enabler** when the work creates technical capability needed for future delivery, but does not by itself deliver direct end-user business value.

Recommended Title Format

Enable <technical capability>

Description Template

The Description field for an Enabler should use the following structure:

Capability to be created or improved:
Why it is needed:
Which future PBIs it enables:
Technical scope:
Explicit exclusions:

Generic Authoring Template

PBI-###: [Sequential ID]
Type: Enabler
Title: Enable <technical capability>

Description:

Capability to be created or improved:
Why it is needed:
Which future PBIs it enables:
Technical scope:
Explicit exclusions:

AcceptanceCriteria:

- Given the capability is implemented, when it is used, then the intended technical outcome is available.
- Given future PBIs depend on the capability, when those teams consume it, then usage guidance exists.
- Given the capability affects operations, when deployed, then runbook and monitoring updates exist.

ReqIDs: Rxx or linked parent requirement

NFR_Tags: Security;Reliability;Maintainability;Observability;Compliance

StoryPoints: [Team estimate]

Priority: P0 / P1 / P2
Owner: [Name / Role]
Sprint: [Blank until planned or Sprint N]
Status: New / Not Ready / Ready for Refinement
Dependencies: [Platform / infra / ADR / external approval]
Notes: [Enabled PBIs / rollout notes / operational considerations]

Detailed Example

PBI-103

Type: Enabler

Title: Enable automated dependency vulnerability scanning in CI

Description:

Capability to be created or improved: Add software composition analysis scanning to PR and protected-branch pipelines.

Why it is needed: Baseline CI security controls require automated dependency checks before merge.

Which future PBIs it enables: All Stories, Bugs, and Enablers that add or update dependencies.

Technical scope: Add CI job, severity thresholds, failure policy, and report publication.

Explicit exclusions: Remediation of all existing vulnerabilities is not included.

AcceptanceCriteria:

- Given a pull request modifies dependencies, when CI runs, then an automated SCA result is generated.
- Given a critical vulnerability is detected, when CI completes, then the pull request is blocked according to policy.
- Given developers need the scan results, when the CI job completes, then a readable vulnerability report is available.

ReqIDs: R41

NFR_Tags: Security;Maintainability;Compliance

StoryPoints: 5

Priority: P1

Owner: DevOps Engineer

Sprint: Sprint 2

Status: Ready for Refinement

Dependencies: CI runner access;approved scanning tool

Notes: Enables secure merge gating for future Stories.

1.6 Bug Template

When to Use

Use a **Bug** when existing system behavior differs from required or expected behavior.

Recommended Title Format

Fix <defect summary>

Description Template

The Description field for a Bug should use the following structure:

Observed behavior:
Expected behavior:
Business or technical impact:
Affected users or environments:
Reproduction steps:

Generic Authoring Template

PBI-###: [Sequential ID]
Type: Bug
Title: Fix <defect summary>

Description:

Observed behavior:
Expected behavior:
Business or technical impact:
Affected users or environments:
Reproduction steps:

AcceptanceCriteria:

- Given the defect scenario, when the relevant action is repeated, then the issue no longer occurs.
- Given related normal behavior, when the fix is applied, then existing functionality still works correctly.
- Given the defect has operational significance, when the event recurs, then diagnostic logging or monitoring is sufficient.

ReqIDs: Rxx or violated parent requirement

NFR_Tags: Reliability;Security;Performance;Audit

StoryPoints: [Team estimate]

Priority: P0 / P1 / P2

Owner: [Name / Role]

Sprint: [Blank until planned or Sprint N]

Status: New / Not Ready / Ready for Refinement

Dependencies: [Incident ticket / external system / parent PBI]

Notes: [Logs / screenshots / suspected cause / incident reference]

Detailed Example

PBI-104
Type: Bug

Title: Fix duplicate supplier record creation on retry submission

Description:

Observed behavior: Retry after timeout may create a second supplier record.

Expected behavior: Submission must be idempotent.

Business or technical impact: Data inconsistency, manual cleanup, and audit risk.

Affected users or environments: Procurement users in staging with possible production exposure.

Reproduction steps: Submit supplier registration, force timeout after persistence, and retry the same payload.

AcceptanceCriteria:

- Given the same supplier registration request is retried, when processed again, then no duplicate supplier record is created.
- Given the original request already succeeded, when the user retries, then the same record identifier is returned or referenced.
- Given duplicate-prevention logic is triggered, when logs are reviewed, then the event is traceable for audit and debugging.

ReqIDs: R12;R27

NFR_Tags: Reliability;Audit

StoryPoints: 3

Priority: P0

Owner: Backend Developer

Sprint: Sprint 2

Status: Ready for Refinement

Dependencies: INC-023

Notes: Suspected cause: missing idempotency key handling.

1.7 Task Template

When to Use

Use a **Task** when the work is a bounded action or deliverable that is too specific to be a Story and too narrow to be an Enabler.

Recommended Title Format

Do <specific action>

Description Template

The Description field for a Task should use the following structure:

Action to be performed:

Why it is needed:

Expected output:

Scope:

Explicit exclusions:

Generic Authoring Template

PBI-###: [Sequential ID]

Type: Task

Title: Do <specific action>

Description:

Action to be performed:

Why it is needed:

Expected output:

Scope:

Explicit exclusions:

AcceptanceCriteria:

- Given the work is completed, when reviewed, then the expected output exists.
- Given the output is intended for use by another role or team, when validated, then it is usable and complete.
- Given the Task supports a parent PBI, when traceability is reviewed, then the linkage is recorded.

ReqIDs: Rxx or inherited from parent PBI

NFR_Tags: [If applicable]

StoryPoints: [Usually small estimate]

Priority: P0 / P1 / P2

Owner: [Name / Role]

Sprint: [Blank until planned or Sprint N]

Status: New / Not Ready / Ready for Refinement

Dependencies: [Parent PBI / environment / approval]

Notes: [Artifact path / constraints / linkage notes]

Detailed Example

PBI-105

Type: Task

Title: Do staging runbook update for supplier onboarding release

Description:

Action to be performed: Update the staging and production runbook for the new supplier onboarding flow.

Why it is needed: QA and Ops require deployment, validation, and rollback guidance before release.

Expected output: Updated runbook committed in the repository.

Scope: Deployment steps, validation checks, rollback steps, and operational contacts.

Explicit exclusions: No infrastructure changes are included.

AcceptanceCriteria:

- Given the onboarding flow is ready for staging, when the runbook is reviewed, then deployment, validation, and rollback steps are documented.

- Given an operator follows the runbook, when executing the checks, then required commands, URLs, and validation steps are present.
- Given traceability is reviewed, when the Task is complete, then the runbook update is linked to the parent Story.

ReqIDs: R12

NFR_Tags: Operability;Documentation

StoryPoints: 2

Priority: P1

Owner: QA Lead

Sprint: Sprint 2

Status: Ready for Refinement

Dependencies: PBI-101

Notes: Store output under docs/runbooks/supplier-onboarding.md.

1.8 Complete PBI Templates and Mock Canonical Backlog Examples

All PBIs must be created in the canonical backlog CSV with the following required fields:

PBI-###, Type, Title, Description, AcceptanceCriteria, ReqIDs, NFR_Tags, StoryPoints, Priority, Owner, Sprint, Status, Dependencies, Notes

The examples below show the full canonical structure rather than a shortened summary.

Description Template by PBI Type

The Description field should be structured according to the nature of the PBI type.

Story — Description Template

Context:

Business problem:

User role / actor:

Expected behavior:

Business value:

In scope:

Out of scope:

Spike — Description Template

Unknown / problem statement:

Why a decision is needed now:

Investigation scope:

Explicit exclusions:

Approach:

Enabler — Description Template

Capability to be created or improved:
Why it is needed:
Which future PBIs it enables:
Technical scope:
Explicit exclusions:

Bug — Description Template

Observed behavior:
Expected behavior:
Business or technical impact:
Affected users or environments:
Reproduction steps:

Task — Description Template

Action to be performed:
Why it is needed:
Expected output:
Scope:
Explicit exclusions:

Mock PBIs in Full Canonical Table Form

Because the canonical backlog contains many fields, the example table is shown in landscape orientation.

PBI ID	Type	Title	Description	AcceptanceCriteria	ReqIDs	NFR_Tags	Story-Points	Prior-ity	Owner
PBI-101	Story	As a procurement officer, I want to submit a supplier evaluation form, so that vendor selection is recorded consistently	Context: supplier evaluations are currently recorded in email and spreadsheets. Business problem: weak audit traceability and inconsistent evidence. User role / actor: procurement officer. Expected behavior: user can complete, validate, and submit a standard form. Business value: improves governance and standardization. In scope: create, validate, submit, persist. Out of scope: approval workflow and analytics.	Given an authenticated procurement officer, when all mandatory fields are completed and submitted, then the evaluation is saved and a reference number is displayed. Given required fields are missing, when submission is attempted, then validation errors are shown and submission is blocked. Given a saved evaluation exists, when an auditor views the record, then submitter identity and timestamp are visible.	R12;R19	Audit;Security;Usability	5	P1	Aisyah
PBI-102	Spike	Investigate PLS profit-sharing calculation engine approach	Unknown / problem statement: it is unclear whether PLS calculation logic should be implemented in application code, a rules engine, or a configurable service. Why a decision is needed now: multiple future PBIs depend on a stable and auditable implementation approach. Investigation scope: compare correctness, traceability, performance, and ease of change. Explicit exclusions: no production implementation. Approach: prototype representative scenarios and evaluate options.	Given the Spike is concluded, when the timebox ends, then a written recommendation is attached. Given at least two design options are reviewed, when the Spike closes, then trade-offs, assumptions, and risks are documented. Given an architectural decision is required, when the Spike closes, then a draft ADR and follow-up PBIs are identified.	R33	Compliance;Auditability;Performance;Architecture	3	P0	Farid
PBI-103	Enabler	Enable automated dependency vulnerability scanning in CI	Capability to be created or improved: add software composition analysis scanning to PR and protected-branch pipelines. Why it is needed: baseline CI security controls require automated dependency checks before merge. Which future PBIs it enables: all Stories, Bugs, and Enablers that add or update dependencies. Technical scope: CI job, severity thresholds, failure policy, report publication. Explicit exclusions: remediation of all existing vulnerabilities.	Given a pull request modifies dependencies, when CI runs, then an SCA result is generated automatically. Given a critical vulnerability is detected, when CI completes, then the pull request is blocked according to policy. Given developers need scan results, when the job finishes, then a readable report is available.	R41	Security;Maintainability;Compliance	5	P1	DevOps Engineer

PBI ID	Type	Title	Description	AcceptanceCriteria	ReqIDs	NFR__Tags	Story-Points	Prior-ity	Owner
PBI-104	Bug	Fix duplicate supplier record creation on retry submission	Observed behavior: retry after timeout may create a second supplier record. Expected behavior: submission must be idempotent. Business or technical impact: data inconsistency, manual cleanup, and audit risk. Affected users or environments: procurement users in staging with possible production exposure. Reproduction steps: submit supplier registration, force timeout after persistence, retry same payload.	Given the same supplier registration request is retried, when processed again, then no duplicate supplier record is created. Given the original request already succeeded, when the user retries, then the same record identifier is returned or referenced. Given duplicate-prevention logic is triggered, when logs are reviewed, then the event is traceable for audit and debugging.	R12;R27	Reliability;Audit	3	P0	Backend Developer
PBI-105	Task	Do staging runbook update for supplier onboarding release	Action to be performed: update the staging and production runbook for the new supplier onboarding flow. Why it is needed: QA and Ops require deployment, validation, and rollback guidance before release. Expected output: updated runbook committed in repository. Scope: deployment steps, validation checks, rollback steps, operational contacts. Explicit exclusions: no infrastructure changes.	Given the onboarding flow is ready for staging, when the runbook is reviewed, then deployment, validation, and rollback steps are documented. Given an operator follows the runbook, when executing the checks, then required commands, URLs, and validation steps are present. Given traceability is reviewed, when the Task is complete, then the runbook update is linked to the parent Story.	R12	Operability;Documentation	2	P1	QA Lead

Mock PBIs as Boxed Canonical CSV

The following example shows the same PBIs exactly as canonical backlog rows.

Example Canonical Backlog CSV

PBI-###,Type,Title,Description,AcceptanceCriteria,ReqIDs,NFR_Tags,StoryPoints,Priority,Owner,Sprint,Status,Dependencies,Notes

PBI-101,Story,"As a procurement officer, I want to submit a supplier evaluation form, so that vendor selection is recorded consistently","Context: supplier evaluations are currently recorded in email and spreadsheets. Business problem: weak audit traceability and inconsistent evidence. User role / actor: procurement officer. Expected behavior: user can complete, validate, and submit a standard form. Business value: improves governance and standardization. In scope: create, validate, submit, persist. Out of scope: approval workflow and analytics .","Given an authenticated procurement officer, when all mandatory fields are completed and submitted, then the evaluation is saved and a reference number is displayed. | Given required fields are missing, when submission is attempted, then validation errors are shown and submission is blocked. | Given a saved evaluation exists, when an auditor views the record, then submitter identity and timestamp are visible.",,"R12;R19","Audit;Security;Usability",5,P1,"Aisyah","Sprint 2","Ready for Refinement","None","Capture audit metadata for compliance review"

PBI-102,Spike,"Investigate PLS profit-sharing calculation engine approach","Unknown / problem statement: it is unclear whether PLS calculation logic should be implemented in application code, a rules engine, or a configurable service. Why a decision is needed now: multiple future PBIs depend on a stable and auditable implementation approach. Investigation scope: compare correctness, traceability, performance, and ease of change. Explicit exclusions: no production implementation. Approach: prototype representative scenarios and evaluate options.",,"Given the Spike is concluded, when the timebox ends, then a written recommendation is attached. | Given at least two design options are reviewed, when the Spike closes, then trade-offs, assumptions, and risks are documented. | Given an architectural decision is required, when the Spike closes, then a draft ADR and follow-up PBIs are identified.",,"R33","Compliance; Auditability;Performance;Architecture",3,P0,"Farid","Sprint 2","Ready for Refinement","PBI -115;ADR-TBD","Timebox: 5 calendar days. Exit Criteria: recommendation, prototype evidence, draft ADR"

PBI-103,Enabler,"Enable automated dependency vulnerability scanning in CI","Capability to be created or improved: add software composition analysis scanning to PR and protected-branch pipelines. Why it is needed: baseline CI security controls require automated dependency checks before merge. Which future PBIs it enables: all Stories, Bugs, and Enablers that add or update dependencies. Technical scope: CI job, severity thresholds, failure policy, report publication. Explicit exclusions: remediation of all existing vulnerabilities.",,"Given a pull request modifies dependencies, when CI runs, then an SCA result is generated automatically. | Given a critical vulnerability is detected, when CI completes, then the pull request is blocked according to policy. | Given developers need scan results, when the job finishes, then a readable report is available.",,"R41","Security;Maintainability;Compliance",5,P1,"DevOps Engineer","Sprint 2","Ready for Refinement","CI runner access;approved scanning tool","Enables secure merge gating for future Stories"

PBI-104,Bug,"Fix duplicate supplier record creation on retry submission","Observed behavior: retry after timeout may create a second supplier record. Expected behavior: submission must be idempotent. Business or technical impact: data inconsistency, manual cleanup, and audit risk. Affected users or environments: procurement users in staging with possible production exposure . Reproduction steps: submit supplier registration, force timeout after persistence, retry same payload.",,"Given the same supplier registration request is retried, when processed again, then no duplicate supplier record is created. | Given the original request already succeeded, when the user retries, then the same record identifier is returned or referenced. | Given duplicate-prevention logic is triggered, when logs are reviewed, then the event is traceable for audit and debugging.",,"R12;R27","Reliability;Audit",3,P0,"Backend Developer","Sprint 2","Ready for Refinement","INC-023","Suspected cause: missing idempotency key handling"

PBI-105,Task,"Do staging runbook update for supplier onboarding release","Action to be performed: update the staging and production runbook for the new supplier onboarding flow. Why it is needed: QA and Ops require deployment, validation, and rollback guidance before release. Expected output: updated runbook committed in repository. Scope: deployment steps, validation checks, rollback steps, operational contacts. Explicit exclusions: no infrastructure changes .","Given the onboarding flow is ready for staging, when the runbook is reviewed, then deployment, validation, and rollback steps are documented. | Given an operator follows the runbook, when executing the checks, then required commands, URLs, and validation steps are present. | Given traceability is reviewed, when the Task is complete, then the runbook update is linked to the parent Story.",,"R12","Operability;Documentation",2,P1,"QA Lead","Sprint 2","Ready for Refinement","PBI-101","Store output under docs/runbooks/supplier-onboarding.md"

Authoring Note

The examples above preserve the same canonical field structure across all PBI types, but the content emphasis differs by type:

- **Story** emphasizes business value and user-facing behavior.
- **Spike** emphasizes uncertainty, exit criteria, and timebox.
- **Enabler** emphasizes technical capability and future delivery support.
- **Bug** emphasizes observed versus expected behavior and reproducibility.
- **Task** emphasizes a bounded action and a concrete output.

1.9 Suggested Stage 1 Authoring Policy Text

The following wording may be inserted directly into the governance guide:

Each PBI type shall use a structured authoring pattern appropriate to the nature of the work. Stories shall express business or user value. Spikes shall express uncertainty reduction and must include explicit exit criteria and a timebox in calendar days. Enablers shall define technical capabilities required for future delivery. Bugs shall state observed versus expected behavior and include reproducible defect context. Tasks shall define a bounded work action and a clear deliverable.

A PBI shall be marked **Not Ready** if it does not contain a meaningful Title, Description, at least one Acceptance Criterion, at least one ReqID reference, and an assigned Owner.